

MLE, MoM and EM for Parameter Estimation

The set up for parameter estimation

For each method, I ran 100 time to see their bias and variance, also check the density function at the same time.

```
knitr::opts_chunk$set(echo = TRUE, warning = FALSE)
knitr::opts_chunk$set(fig.width=7, fig.height=4)
library(rootSolve)

# Inverse Guassian random number
IG_random = function(n, lambda, theta) {
  u = runif(n, 0, 1)
  z = rnorm(n, 0, 1)
  v = lambda * theta + (theta / 2) * (z^2 - sqrt(z^4 + 4 * lambda * z^2))
  compare = (lambda * theta) / (lambda * theta + v)
  index = u < compare
  another_choice = (lambda^2) * (theta^2) / v
  IG = c(v[index], another_choice[!index])
  return(sample(IG))
}

#Length Biased Inverse Gaussian random number
LB_random = function(n, lambda, theta) {
  z2 = rnorm(n, 0, 1)
  IG = IG_random(n, lambda, theta)
  LB = IG + theta * z2^2
  return(sample(LB))
}

#Crack random number
Crack_random = function(n, lambda, theta, p) {
  IG = IG_random(n, lambda, theta)
  LB = LB_random(n, lambda, theta)
  u = runif(n, 0, 1)
  index = u < p
  Crack = c(IG[index], LB[!index])
  return(sample(Crack))
}

#Density function of Twice Length Biased Inverse Gaussian
f_LB2 = function(x, lambda, theta) {
  first_part = sqrt(x / theta) / (theta * sqrt(2*pi) * (lambda + 1))
  second_part = (-1/2) * (sqrt(x/theta) - lambda * sqrt(theta/x))^2
  return(first_part*(exp(second_part)))
}
```

```

#Twice Length Biased Inverse Gaussian random number
LB2_random = function(n, lambda, theta) {
  LB2 = c()
  while(length(LB2) < n) {
    u = runif(n, 0, 1)
    gamma = rgamma(n, shape = 3/2, scale = 2*theta)
    compare = f_LB2(gamma, lambda, theta) / (exp(lambda) * dgamma(gamma, shape = 3/2, scale = 2*theta))
    index = u <= compare
    LB2 = c(LB2, gamma[index])
  }
  return(sample(LB2)[1:n])
}

#Generalized Crack random number
GCR_random = function(n, lambda, theta, p, q) {
  r = 1 - p - q
  u = runif(n, 0, 1)
  Crack = Crack_random(n, lambda, theta, p / (1 - r))
  LB2 = LB2_random(n, lambda, theta)
  index = u <= (1 - r)
  GCR = c(Crack[index], LB2[!index])
  return(sample(GCR))
}

#Density function of Generalized Crack
f_GCR = function(x, lambda, theta, p, q) {
  r = 1 - p - q
  first_part = sqrt(theta/(2*pi))
  second_part = (p*lambda*(x^(-3/2)) + (q/theta)*x^(-1/2) + (r/((lambda + 1)*theta^(2)))*x^(1/2))
  third_part = (-1/2) * (sqrt(x/theta) - lambda*(sqrt(theta/x)))^2
  return(first_part * second_part * exp(third_part))
}

#Density function of Inverse Gaussian
f_IG = function(x, lambda, theta) {
  return(lambda*sqrt(theta/(2*pi))*x**(-3/2)*exp(-(x - lambda*theta)**2/(2*theta*x)))
}

#density function of Length Biased Inverse Gaussian
f_LB = function(x, lambda, theta) {
  return(1/(theta*sqrt(2*pi))*sqrt(theta/x)*exp((-1/2)*(sqrt(x/theta) - lambda*sqrt(theta/x))**2))
}

```

MLE method

The method is exact the same as before.

```

MLE_log = function(lambda, theta, p, q, x) {
  n = length(x)
  r = 1 - q - p
  first = (n / 2) * (log(theta) - log(2*pi))

```

```

second = p * lambda * (x**(-3/2)) + (q/theta) * (x**(-1/2)) + (r/((lambda +1)*(theta**2)))*(x**(1/2))
second = sum(log(second))
third = n*lambda - (1/2)*sum(x/theta + (lambda**2)*theta/x)
return(first + second + third)
}

MLE_lt = function(p, q, x) {

  l = function(paras) {
    lambda = paras[1]
    theta = paras[2]
    return(-1*MLE_log(lambda, theta, p = p, q = q, x = x))
  }
  res = optim(par = c(1, 1), fn = l, method = "L-BFGS-B", lower = 0, upper = Inf)
  return(res)

}

# MLE_helper is function to compute lambda, theta and mle according to all combinations of p and q in
MLE_helper = function(grid, x) {
  m = nrow(grid)
  res = data.frame(matrix(0, m, 3))
  colnames(res) = c("lambda", "theta", "mle")
  for (i in 1:m) {
    p = grid[i, "p"]
    q = grid[i, "q"]
    curr_res = MLE_lt(p, q, x)
    res[i, "lambda"] = curr_res$par[1]
    res[i, "theta"] = curr_res$par[2]
    res[i, "mle"] = -curr_res$value
  }
  grid = cbind(grid, res)
  target = which.max(grid[, "mle"])
  ret = list(lambda=grid[target, "lambda"], theta=grid[target, "theta"], p=grid[target, "p"], q=grid[target, "q"])

  return(ret)
}

# grid_maker is a function that list out all possible combinations of q and p, and put them in a grid.
grid_maker = function(p_center, q_center, delta, bins) {
  grid = data.frame(matrix(0, 0, 2))
  colnames(grid) = c("p", "q")
  for (i in -bins:bins) {
    p = p_center + i * delta
    for (j in -bins:bins) {
      q = q_center + j * delta
      if (p >= 0 && p <= 1 && q >= 0 && q <= 1 && p + q <= 1) {
        grid = rbind(grid, data.frame(p = p, q = q))
      }
    }
  }
}

```

```

    return(grid)
}

# MLE_test is a function to implement round 3, with the help of MLE_helper and grid_maker functions.
MLE_onetime = function(n = 1000, true_lambda = 3, true_theta = 2, true_p = 0.3, true_q = 0.2, bins = 5)
{
  x = GCR_random(n, true_lambda, true_theta, true_p, true_q)
  p_center = 0.5
  q_center = 0.5
  delta = 0.5 / bins
  while (delta >= 1e-4) {
    grid = grid_maker(p_center, q_center, delta, bins)
    if (nrow(grid) == 0) {
      break
    }
    choice = MLE_helper(grid, x)
    p_center = choice$p
    q_center = choice$q
    delta = delta/bins - 1e5
  }
  return(choice)
}

MLE_test = function(n = 1000, true_lambda = 3, true_theta = 2, true_p = 0.3, true_q = 0.2, bins = 5) {
  l_arr = c()
  t_arr = c()
  p_arr = c()
  q_arr = c()

  for (i in 1:100) {
    res = MLE_onetime(n, true_lambda, true_theta, true_p, true_q, bins)
    l_arr = c(l_arr, res$lambda)
    t_arr = c(t_arr, res$theta)
    p_arr = c(p_arr, res$p)
    q_arr = c(q_arr, res$q)
  }
  n = length(l_arr)
  ret = list(l_arr = l_arr, t_arr = t_arr, p_arr = p_arr, q_arr = q_arr, lambda = sum(l_arr/n), theta =
  return(ret)
}

```

We test the case $n = 1000$, $\lambda = 3$, $\theta = 2$, $p = 0.4$, $q = 0.4$.

```

set.seed(2021)
MLE_results = MLE_test(n = 1000, true_lambda = 3, true_theta = 2, true_p = 0.4, true_q = 0.4)

```

First we check the value of each parameter:

```

print(paste("lambda is:", MLE_results$lambda))

```

```
## [1] "lambda is: 3.0013907167735"
```

```
print(paste("theta is:", MLE_results$theta))
```

```
## [1] "theta is: 2.00658578827578"
```

```
print(paste("p is:", MLE_results$p))
```

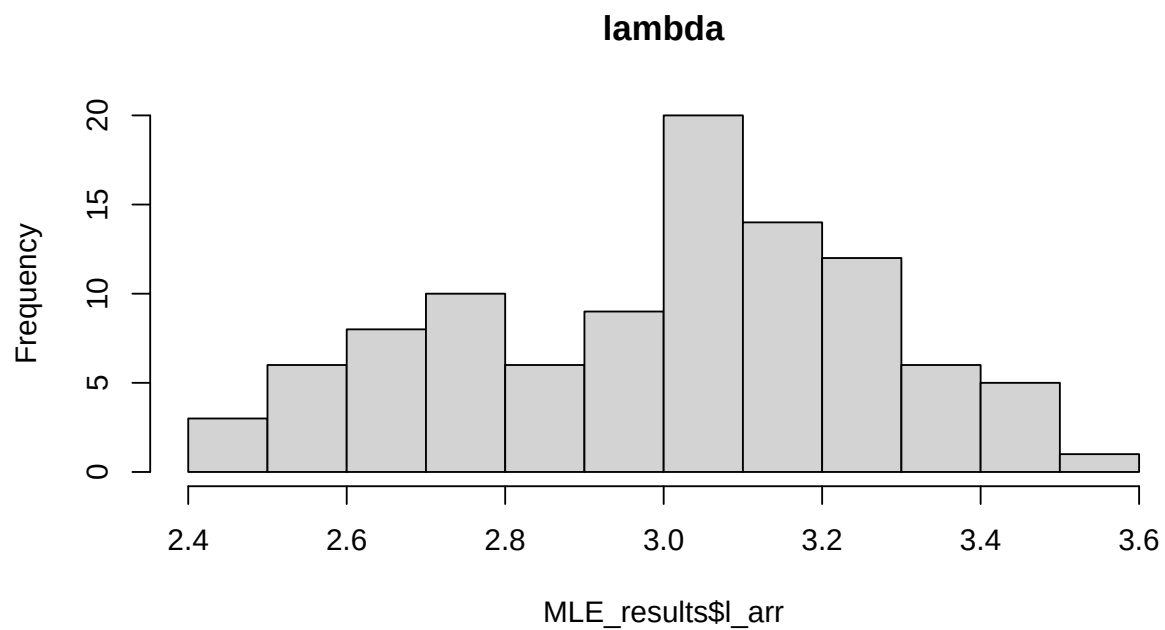
```
## [1] "p is: 0.45"
```

```
print(paste("q is:", MLE_results$q))
```

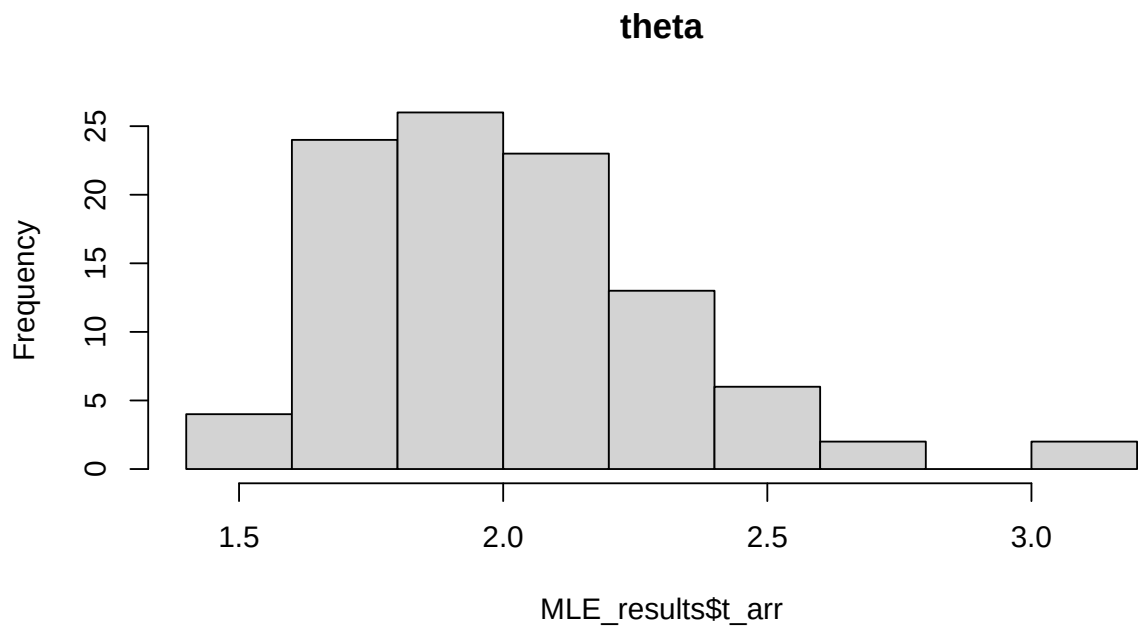
```
## [1] "q is: 0.26"
```

Then we check the distribution of each parameter:

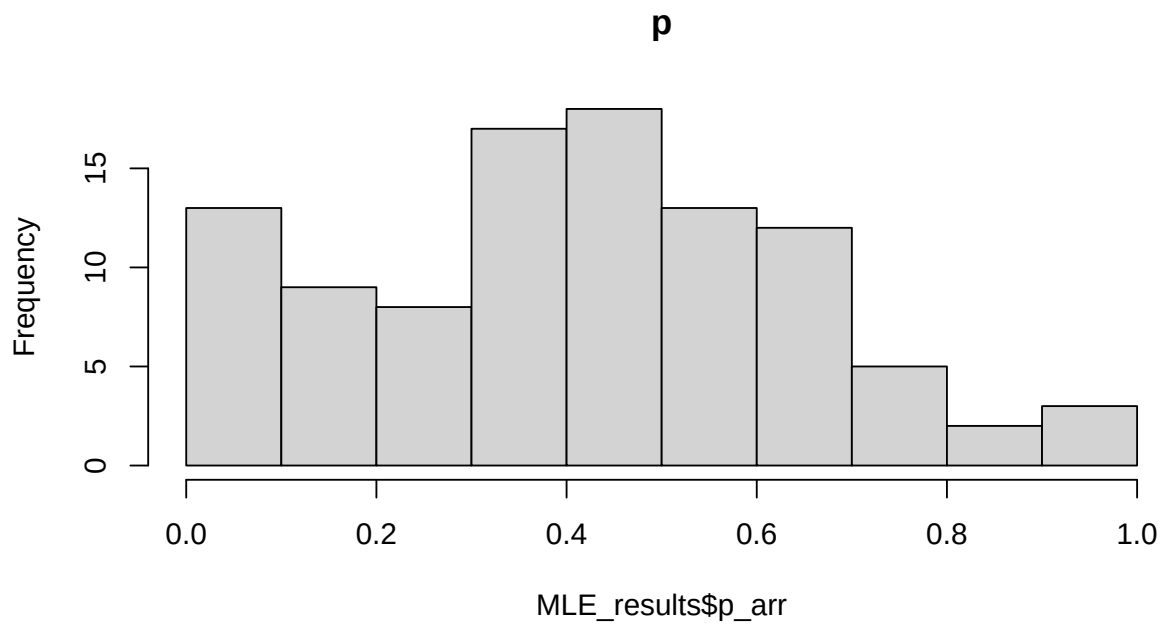
```
hist(MLE_results$l_arr, main = "lambda")
```



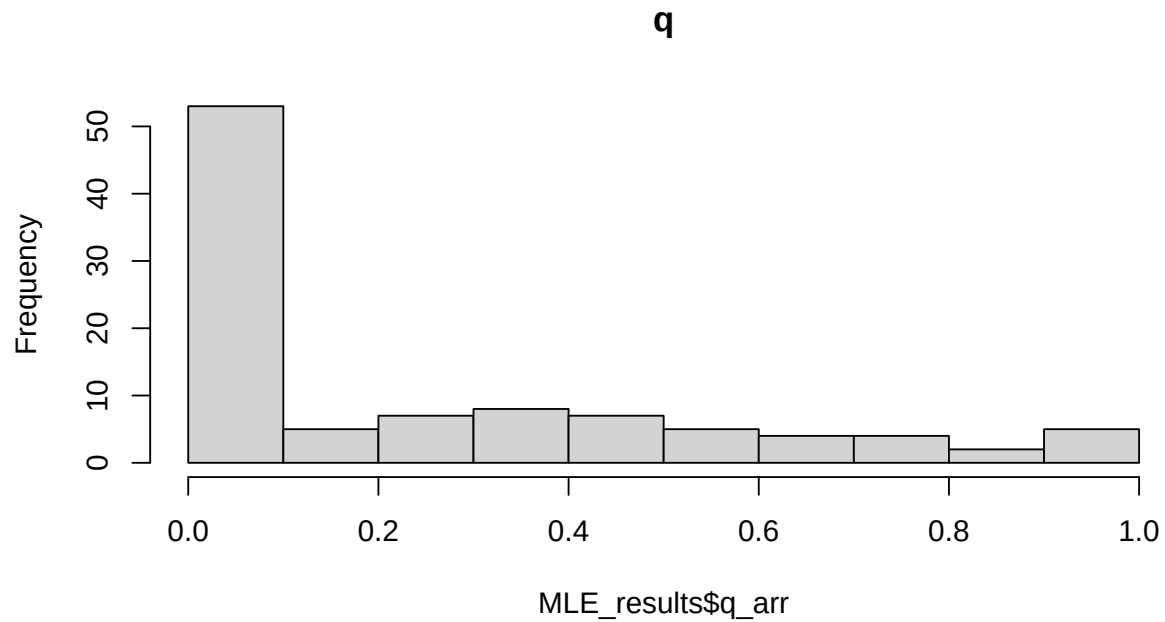
```
hist(MLE_results$t_arr, main = "theta")
```



```
hist(MLE_results$p_arr, main = "p")
```

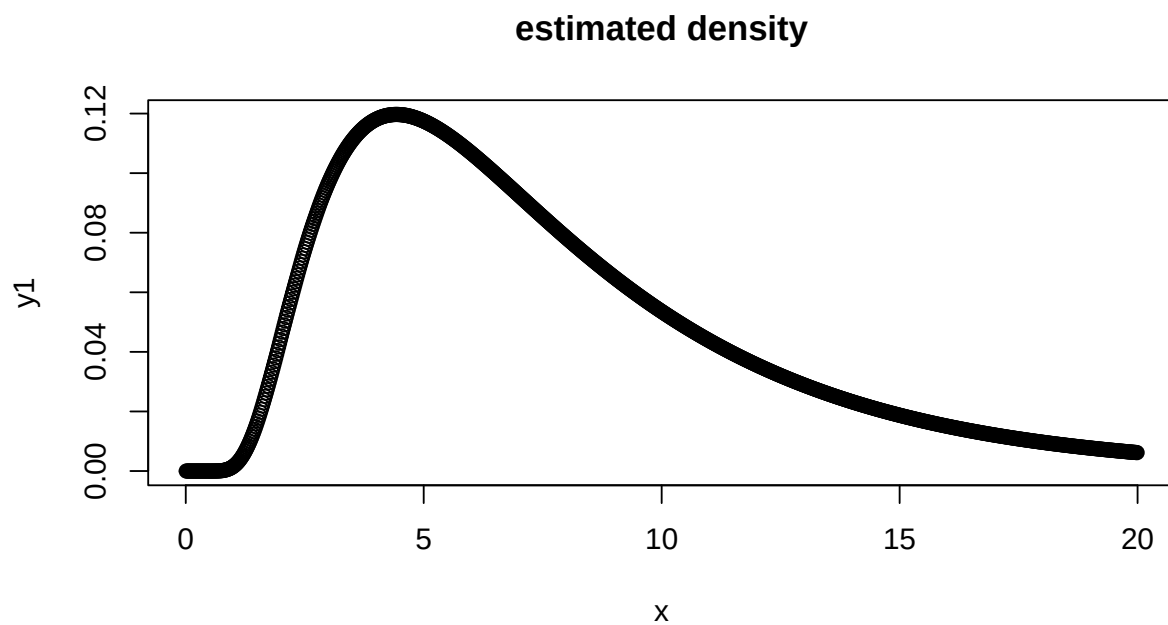


```
hist(MLE_results$q_arr, main = "q")
```

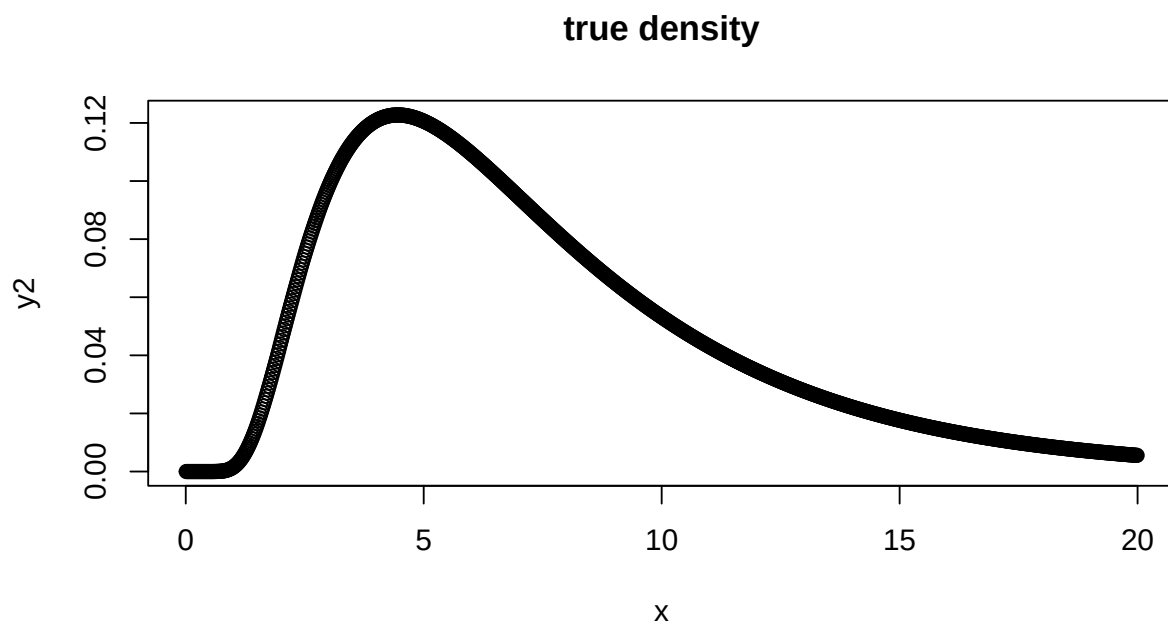


Last we compare the true density function with the estimated one:

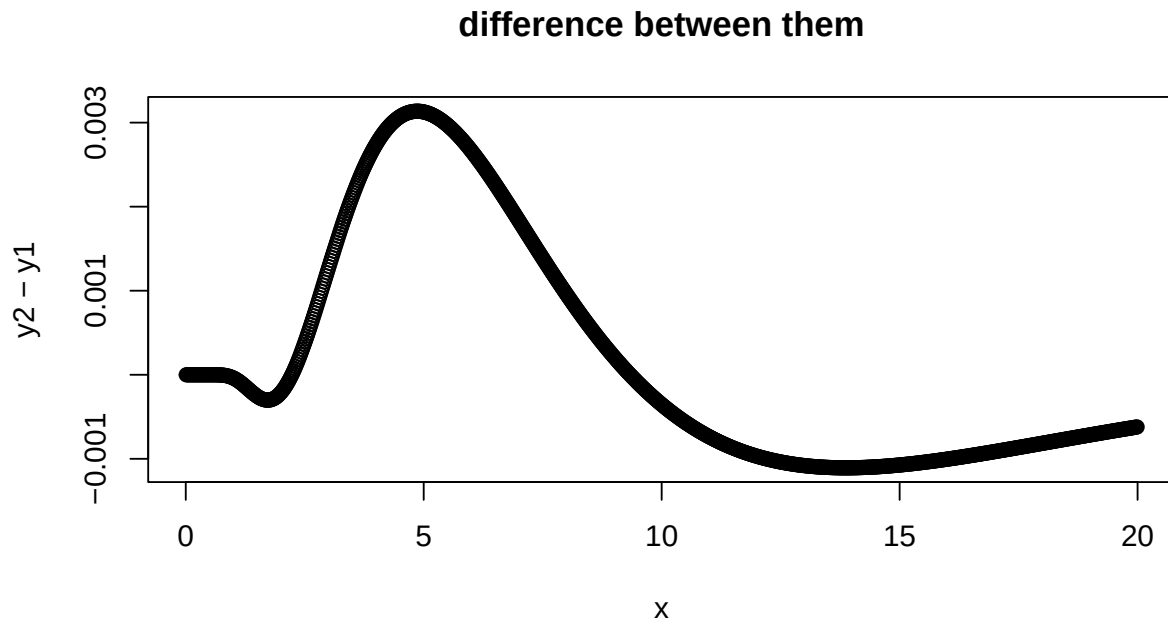
```
set.seed(2021)
x = seq(0.01, 20, 0.02)
y1 = f_GCR(x, MLE_results$lambda, MLE_results$theta, MLE_results$p, MLE_results$q)
y2 = f_GCR(x, 3, 2, 0.4, 0.4)
plot(x, y1, main = "estimated density")
```



```
plot(x, y2, main = "true density")
```

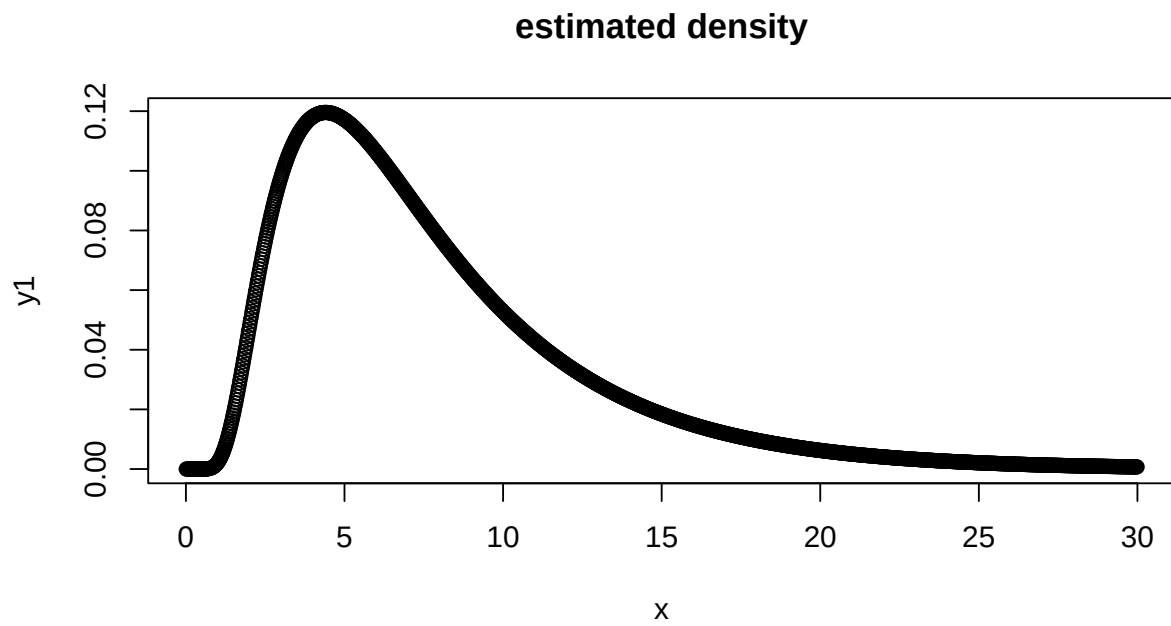



```
plot(x, y2 - y1, main = "difference between them")
```

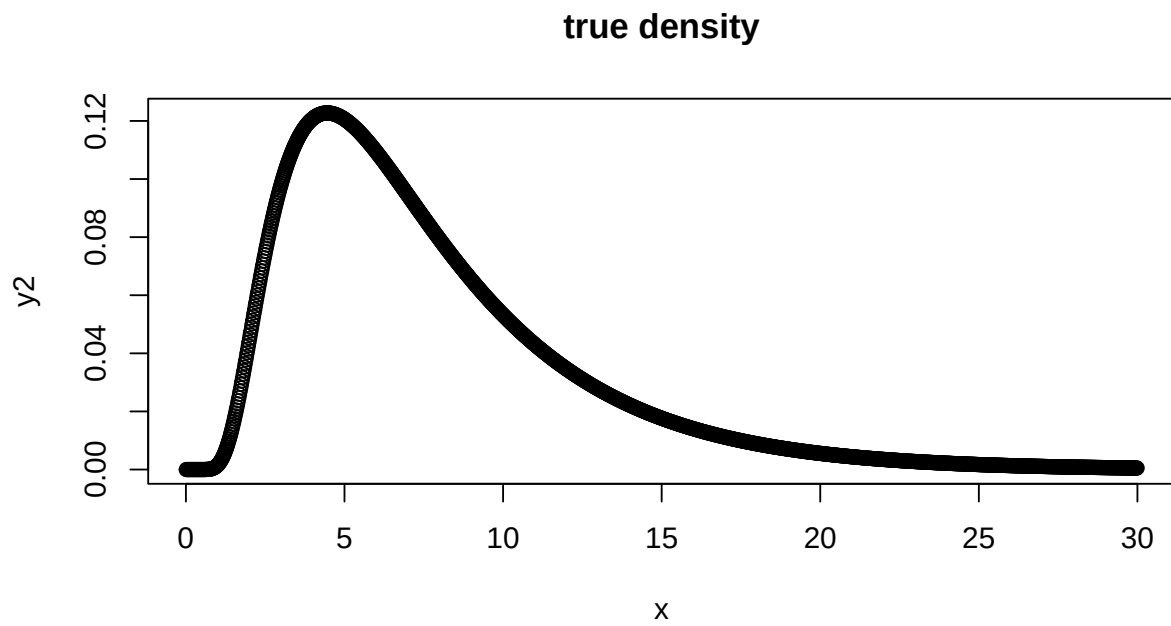


In conclusion, MLE method has high variance and can have high bias as well, and estimated density function and the true one are quite the same. In addition actually if simply choose a result from all the MLE results, the difference between two is almost the same.

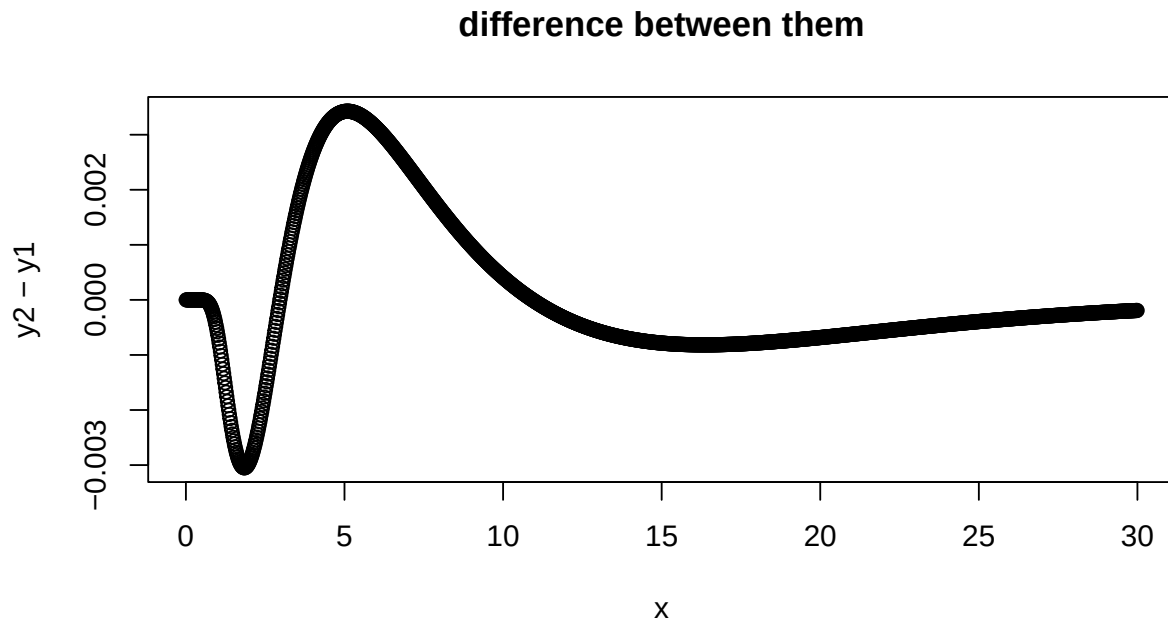
```
set.seed(20212)
x = seq(0.01, 30, 0.02)
choice = sample(1:100, 1)
y1 = f_GCR(x, MLE_results$l_arr[choice], MLE_results$t_arr[choice], MLE_results$p_arr[choice], MLE_results$g_arr[choice])
y2 = f_GCR(x, 3, 2, 0.4, 0.4)
plot(x, y1, main = "estimated density")
```



```
plot(x, y2, main = "true density")
```



```
plot(x, y2 - y1, main = "difference between them")
```



Method of moments

I tried two ways to solve this problem. One is to solve the system of nonlinear algebraic equations numerically, and the other is to minimize the sum of square of difference between estimated moments and real ones.

#The first moment

```
MoM_1 = function(lambda, theta, p, q) {
  r = 1 - p - q
  first = q*theta + r*theta/(2*(lambda + 1)) + 2*r*theta*(1/2 + lambda)/(lambda + 1)
  second = lambda*theta*(p + q + r*(1/2 + lambda)/(lambda + 1))
  return(first + second)
}
```

#The second moment

```
MoM_2 = function(lambda, theta, p, q) {
  r = 1 - p - q
  first = 3*q*theta**2 + 7*r*theta**2/(2*(lambda + 1)) + 8*r*theta**2*(1/2 + lambda)/(lambda + 1)
  second = lambda*theta**2*(p + q + r*(1/2 + lambda)/(lambda + 1)) + lambda**2*theta**2*(p + q + r*(1/2 + lambda)/(lambda + 1))
  third = 2*lambda*theta*(q*theta + r*theta/(2*(lambda + 1)) + 2*r*theta*(1/2 + lambda)/(lambda + 1))
  return(first + second + third)
}
```

#The third moment

```
MoM_3 = function(lambda, theta, p, q) {
  r = 1 - p - q
  first = 15*q*theta**3 + 57*r*theta**3/(2*(lambda + 1)) + 48*r*theta**3*(1/2 + lambda)/(lambda + 1)
```

```

second = (3*lambda*theta**3 + 3*lambda**2*theta**3 + lambda**3*theta**3)*(p + q + r*(1/2 + lambda)/(1
third = (3*lambda*theta**2 + 3*lambda**2*theta**2)*(q*theta + r*theta/(2*(lambda + 1)))
fourth = 3*lambda*theta*(3*q*theta**2 + 7*r*theta**2/(2*(lambda + 1)) + 8*r*theta**2*(1/2 + lambda)/(
return(first + second + third + fourth)
}

#The fourth moment
MoM_4 = function(lambda, theta, p, q) {
  r = 1 - p - q
  first = 105*q*theta**4 + 561*r*theta**4/(2*(lambda + 1)) + 384*r*theta**4*(1/2 + lambda)/(lambda + 1)
  second = (15*lambda*theta**4 + 15*lambda**2*theta**4 + 6*lambda**3*theta**4 + lambda**4*theta**4) * (
  third = (12*lambda*theta**3 + 12*lambda**2*theta**3 + 4*lambda**3*theta**3) * (q*theta + r*theta/(2*(
  return(first + second + third)
}

#solve the system of nonlinear algebraic equations numerically
MoM_onetime1 = function(n = 100, lambda = 2, theta = 3, p = 0.3, q = 0.2) {
  x = GCR_random(n, lambda, theta, p, q)
  M1 = sum(x/n)
  M2 = sum(x**2/n)
  M3 = sum(x**3/n)
  M4 = sum(x**4/n)

  f = function(paras) {
    f1 = MoM_1(paras[1], paras[2], p = paras[3], q = paras[4]) - M1
    f2 = MoM_2(paras[1], paras[2], p = paras[3], q = paras[4]) - M2
    f3 = MoM_3(paras[1], paras[2], p = paras[3], q = paras[4]) - M3
    f4 = MoM_4(paras[1], paras[2], p = paras[3], q = paras[4]) - M4
    return(c(F1 = f1, F2 = f2, F3 = f3, F4 = f4))
  }
  res = multiroot(f, c(4, 3, 0.4, 0.4))
  res = list(lambda = res$root[1], theta = res$root[2], p = res$root[3], q = res$root[4])

  return(res)
}

#Solve by minimizing the sum of square of difference between estimated moments and real ones.
MoM_onetime2 = function(n = 100, lambda = 2, theta = 3, p = 0.3, q = 0.2) {
  x = GCR_random(n, lambda, theta, p, q)
  M1 = sum(x/n)
  M2 = sum(x**2/n)
  M3 = sum(x**3/n)
  M4 = sum(x**4/n)
  f = function(paras) {
    return((MoM_1(paras[1], paras[2], p = paras[3], q = paras[4]) - M1)**2 + (MoM_2(paras[1], paras[2],
      (MoM_3(paras[1], paras[2], p = paras[3], q = paras[4]) - M3)**2 + (MoM_4(paras[1], paras[2],
  }
  res = optim(par = c(4, 3, 0.2, 0.4), fn = f, method = "L-BFGS-B", lower = 0, upper = Inf)
  res = list(lambda = res$par[1], theta = res$par[2], p = res$par[3], q = res$par[4])
  return(res)
}

```

```

#solve the system of nonlinear algebraic equations numerically
MoM_test1 = function(n = 1000, true_lambda = 3, true_theta = 2, true_p = 0.3, true_q = 0.2) {
  l_arr = c()
  t_arr = c()
  p_arr = c()
  q_arr = c()

  for (i in 1:100) {
    res = MoM_onetime1(n, true_lambda, true_theta, true_p, true_q)
    l_arr = c(l_arr, res$lambda)
    t_arr = c(t_arr, res$theta)
    p_arr = c(p_arr, res$p)
    q_arr = c(q_arr, res$q)
  }
  n = length(l_arr)
  ret = list(l_arr = l_arr, t_arr = t_arr, p_arr = p_arr, q_arr = q_arr, lambda = sum(l_arr/n), theta =
  return(ret)
}

#Solve by minimizing the sum of square of difference between estimated moments and real ones.
MoM_test2 = function(n = 1000, true_lambda = 3, true_theta = 2, true_p = 0.3, true_q = 0.2) {
  l_arr = c()
  t_arr = c()
  p_arr = c()
  q_arr = c()

  for (i in 1:100) {
    res = MoM_onetime2(n, true_lambda, true_theta, true_p, true_q)
    l_arr = c(l_arr, res$lambda)
    t_arr = c(t_arr, res$theta)
    p_arr = c(p_arr, res$p)
    q_arr = c(q_arr, res$q)
  }
  n = length(l_arr)
  ret = list(l_arr = l_arr, t_arr = t_arr, p_arr = p_arr, q_arr = q_arr, lambda = sum(l_arr/n), theta =
  return(ret)
}

```

We test the case $n = 1000$, $\lambda = 3$, $\theta = 2$, $p = 0.4$, $q = 0.4$

For results in MoM_results1

First we check the value of each parameter:

```
print(paste("lambda is:", MoM_results1$lambda))
```

```
## [1] "lambda is: NaN"
```

```
print(paste("theta is:", MoM_results1$theta))
```

```
## [1] "theta is: NaN"
```

```
print(paste("p is:", MoM_results1$p))
```

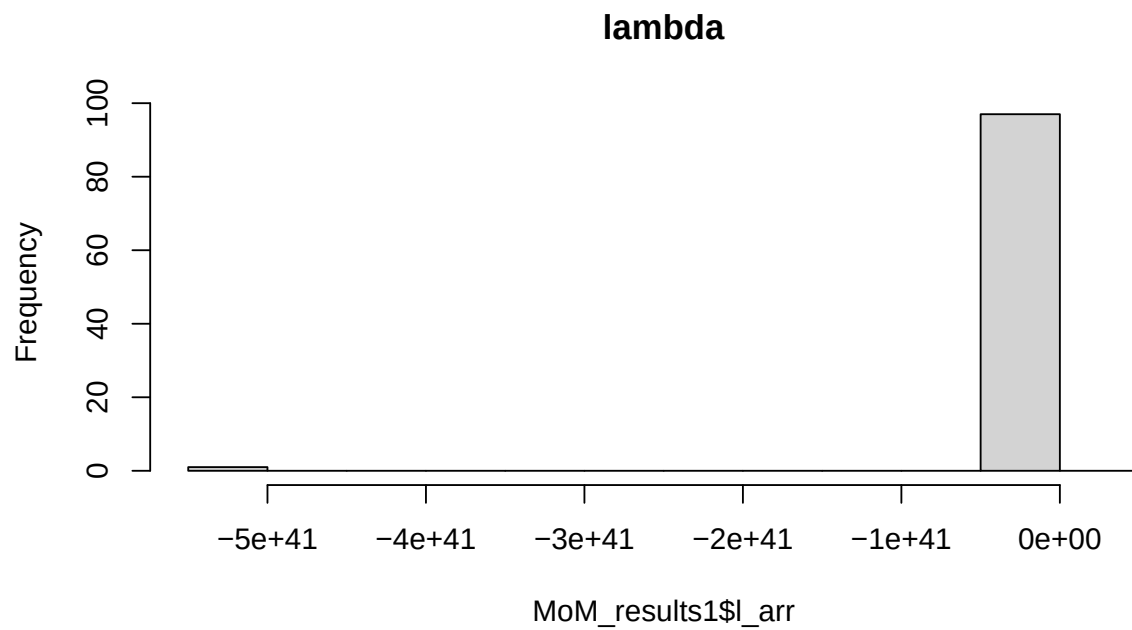
```
## [1] "p is: NaN"
```

```
print(paste("q is:", MoM_results1$q))
```

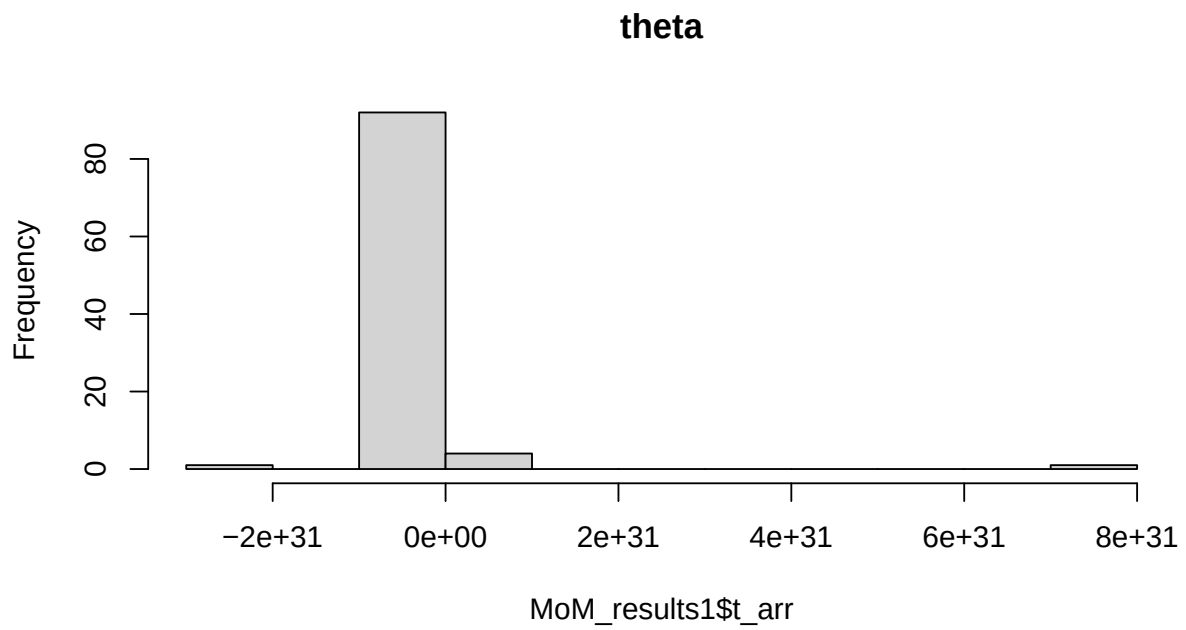
```
## [1] "q is: NaN"
```

Then we check the distribution of each parameter:

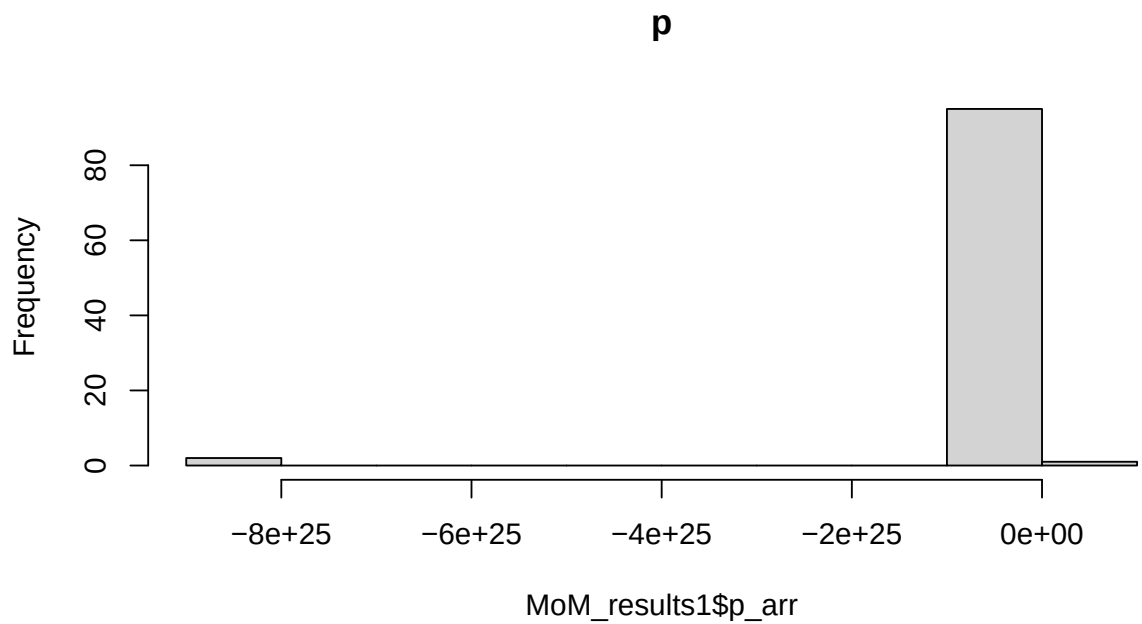
```
hist(MoM_results1$l_arr, main = "lambda")
```



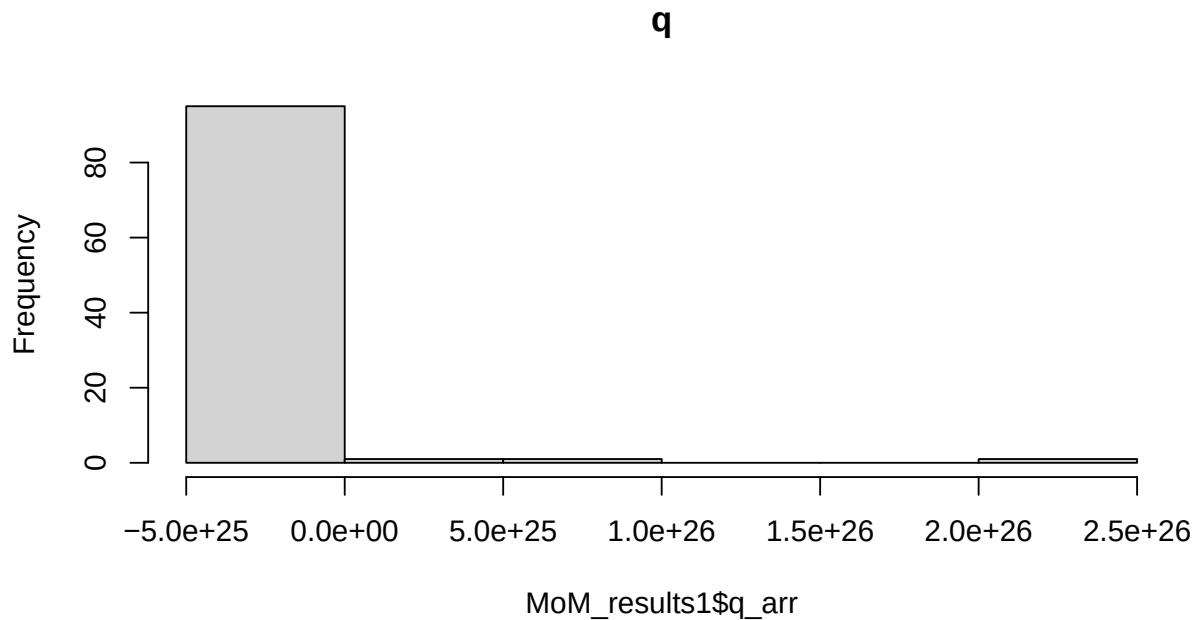
```
hist(MoM_results1$t_arr, main = "theta")
```



```
hist(MoM_results1$p_arr, main = "p")
```



```
hist(MoM_results1$q_arr, main = "q")
```



For results in MoM_results2

First we check the value of each parameter:

```
print(paste("lambda is:", MoM_results2$lambda))
```

```
## [1] "lambda is: 3.36091698532678"
```

```
print(paste("theta is:", MoM_results2$theta))
```

```
## [1] "theta is: 2.74435103396782"
```

```
print(paste("p is:", MoM_results2$p))
```

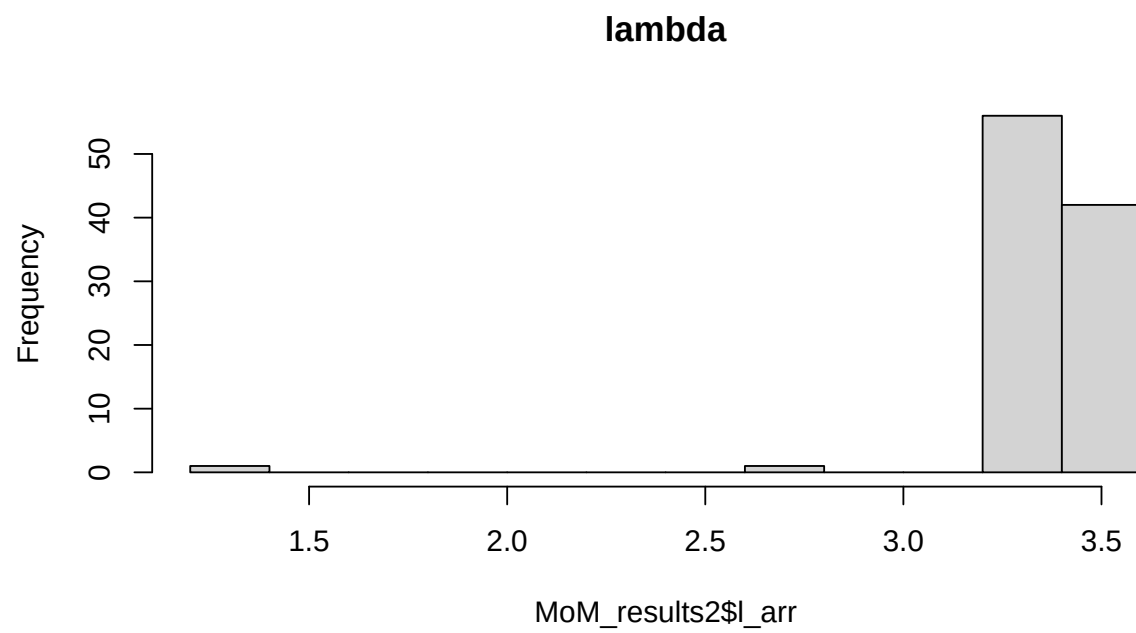
```
## [1] "p is: 1.19727449350019"
```

```
print(paste("q is:", MoM_results2$q))
```

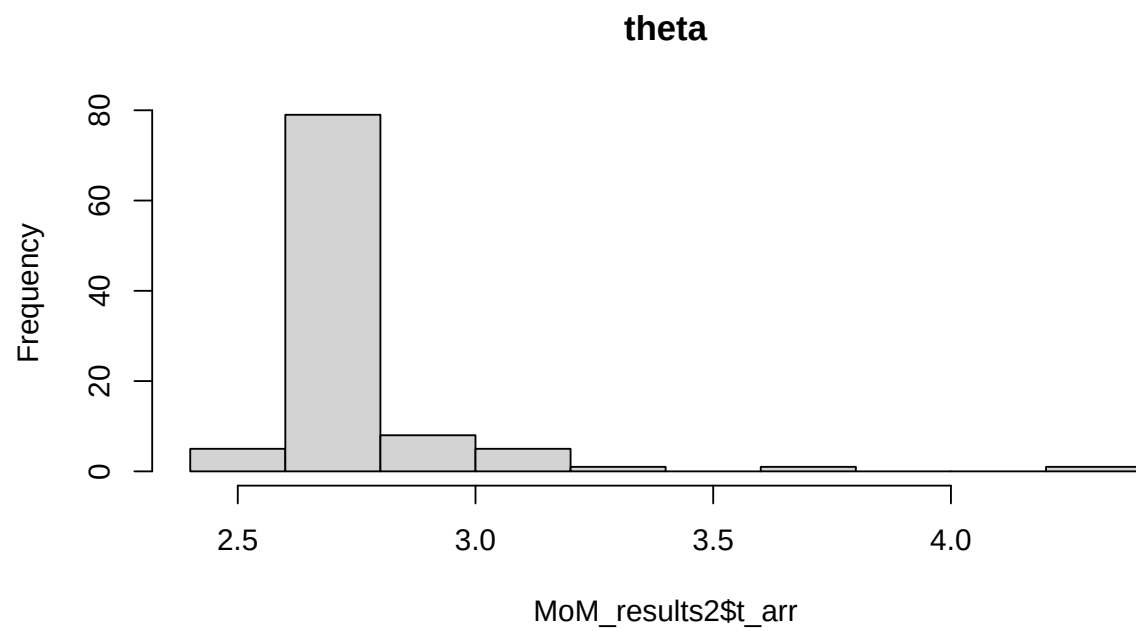
```
## [1] "q is: 0.057463764089829"
```

Then we check the distribution of each parameter:

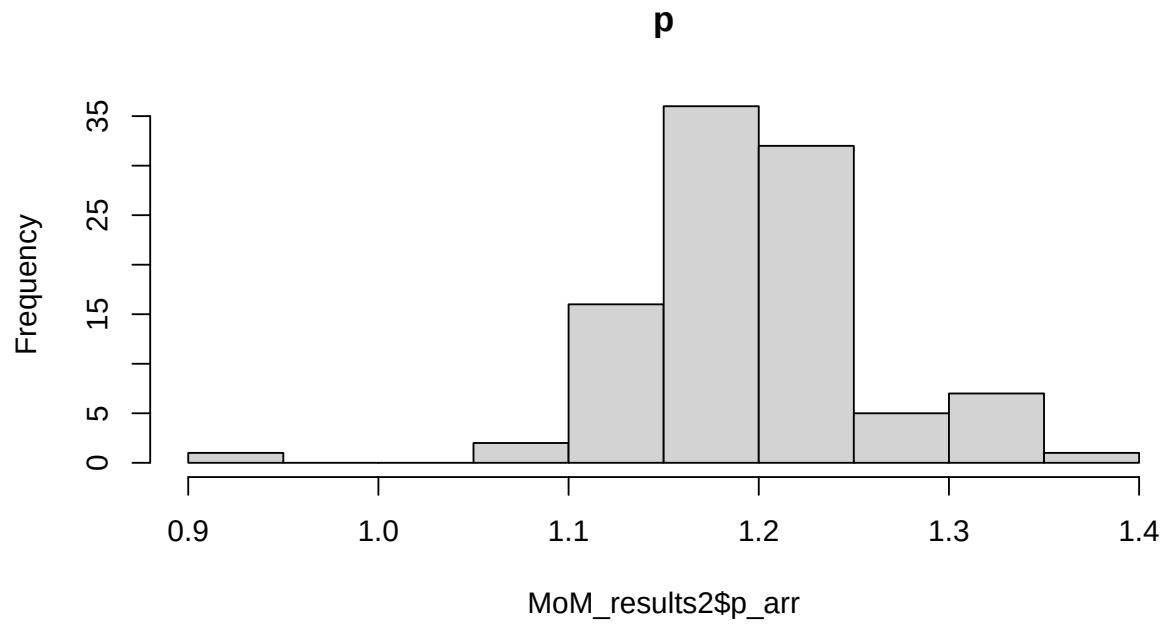

```
hist(MoM_results2$l_arr, main = "lambda")
```



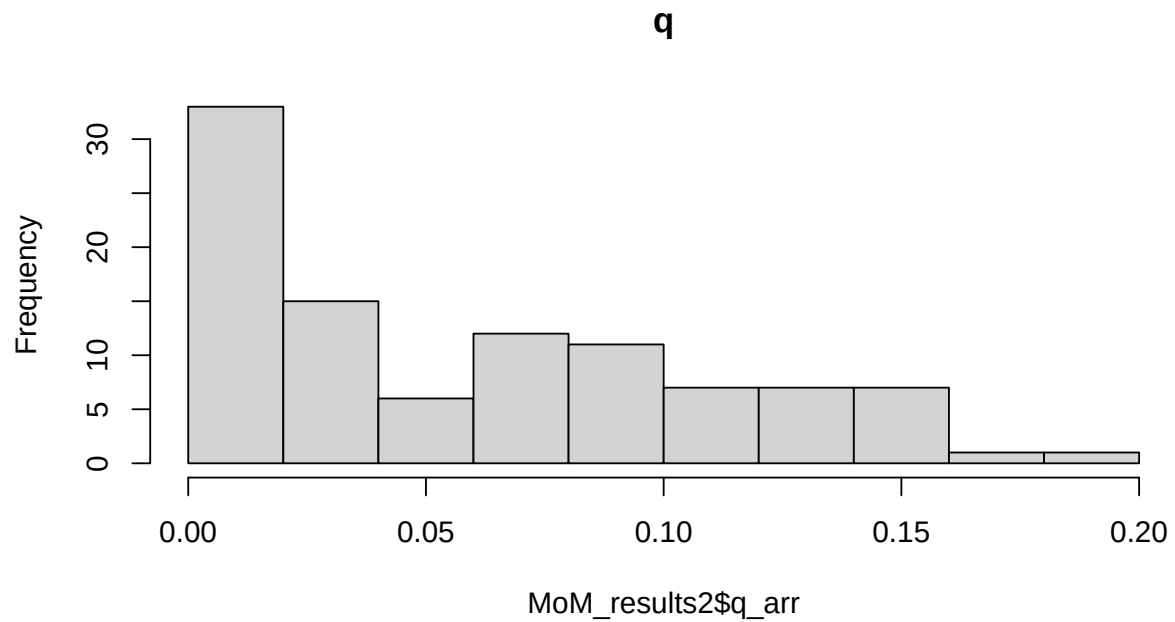
```
hist(MoM_results2$t_arr, main = "theta")
```



```
hist(MoM_results2$p_arr, main = "p")
```



```
hist(MoM_results2$q_arr, main = "q")
```



Both ways to solve Method of moments problem are not appropriate.

EM method

```
EM_onetime = function(x, lambda, theta, p, q) {
  r = 1 - p - q
  de = f_IG(x, lambda, theta)*p + f_LB(x, lambda, theta)*q + f_LB2(x, lambda, theta)*r
  mle = function(l_p, t_p) {
    E = sum((f_IG(x, lambda, theta)*p*log(f_IG(x, l_p, t_p)) +
             f_LB(x, lambda, theta)*p*log(f_LB(x, l_p, t_p)) +
             f_LB2(x, lambda, theta)*p*log(f_LB2(x, l_p, t_p)))/
            de)
    return(E)
  }

  mle_lt = function(paras) {
    return(-mle(paras[1], paras[2]))
  }
  next_lt = optim(par = c(lambda, theta), fn = mle_lt, method = "L-BFGS-B", lower = 0, upper = Inf)$par

  re_0 = sum(f_IG(x, lambda, theta) * p / de) #a
  re_1 = sum(f_LB(x, lambda, theta) * q / de) #c
  re_2 = sum(f_LB2(x, lambda, theta) * r / de) #b
  next_p = re_0*re_2/(re_2*re_1 + re_2**2 + re_0*re_2)
  next_q = re_2*re_1/(re_2*re_1 + re_2**2 + re_0*re_2)

  res = list(lambda = next_lt[1], theta = next_lt[2], p = next_p, q = next_q)

  return(res)
}

EM_test = function(n = 1000, true_lambda = 3, true_theta = 2, true_p = 0.3, true_q = 0.2) {
  l_arr = c()
  t_arr = c()
  p_arr = c()
  q_arr = c()
  for (k in 1:100) {
    x = GCR_random(n, true_lambda, true_theta, true_p, true_q)
    lambda = 2.5
    theta = 2.5
    p = 0.4
    q = 0.2
    for (i in 1:100) {
      res = EM_onetime(x, lambda, theta, p, q)
      next_l = res$lambda
      next_t = res$theta
      next_p = res$p
      next_q = res$q
      if (abs(next_l - lambda) + abs(next_t - theta) + abs(next_p - p) + abs(q - next_q) < 1e-4) {
        break
      }
      lambda = next_l
      theta = next_t
      p = next_p
    }
  }
}
```

```

    q = next_q
  }
  l_arr = c(l_arr, lambda)
  t_arr = c(t_arr, theta)
  p_arr = c(p_arr, p)
  q_arr = c(q_arr, q)
}
n = length(l_arr)
res = list(l_arr = l_arr, t_arr = t_arr, p_arr = p_arr, q_arr = q_arr, lambda = sum(l_arr/n),
          theta = sum(t_arr/n), p = sum(p_arr/n), q = sum(q_arr/n))
return(res)
}

```

We test the case $n = 1000$, $\lambda = 3$, $\theta = 2$, $p = 0.4$, $q = 0.4$

```

set.seed(2021)
EM_results = EM_test(n = 1000, true_lambda = 3, true_theta = 2, true_p = 0.4, true_q = 0.4)

```

```

print(paste("lambda is:", EM_results$lambda))

```

```

## [1] "lambda is: 3.12863225920267"

```

```

print(paste("theta is:", EM_results$theta))

```

```

## [1] "theta is: 1.88010296540925"

```

```

print(paste("p is:", EM_results$p))

```

```

## [1] "p is: 0.450303166285253"

```

```

print(paste("q is:", EM_results$q))

```

```

## [1] "q is: 0.202173490003614"

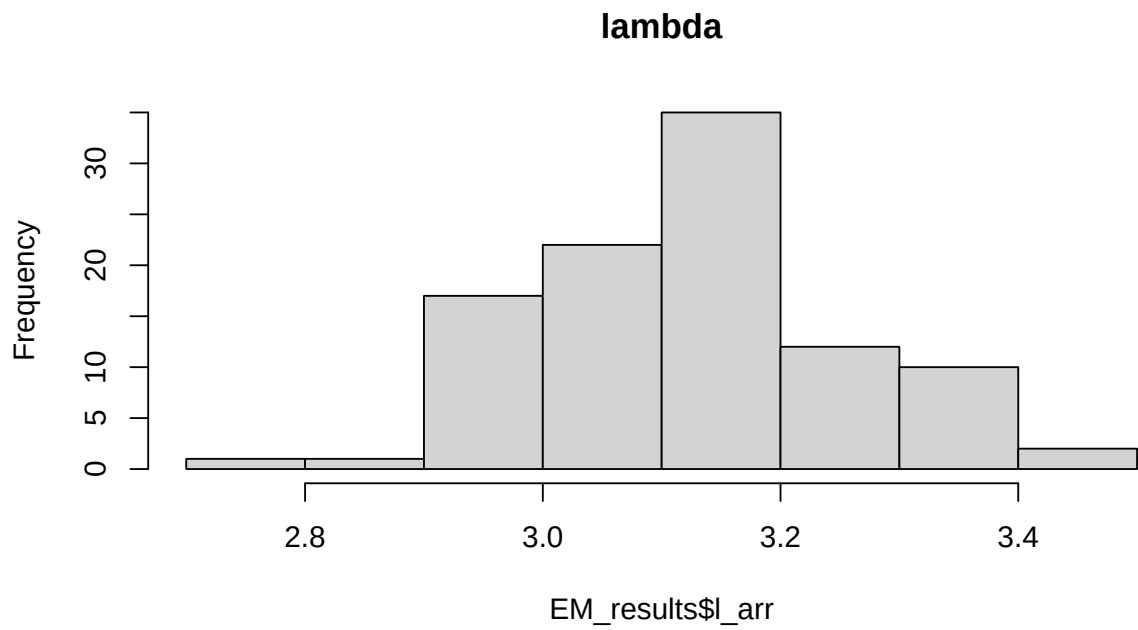
```

Then we check the distribution of each parameter:

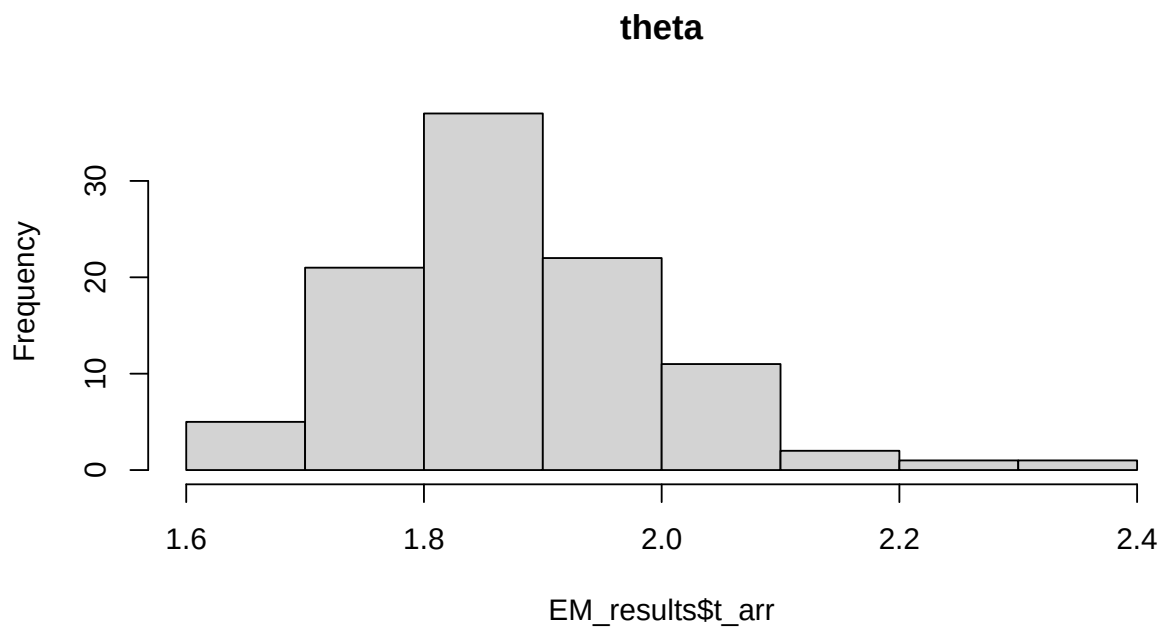
```

hist(EM_results$l_arr, main = "lambda")

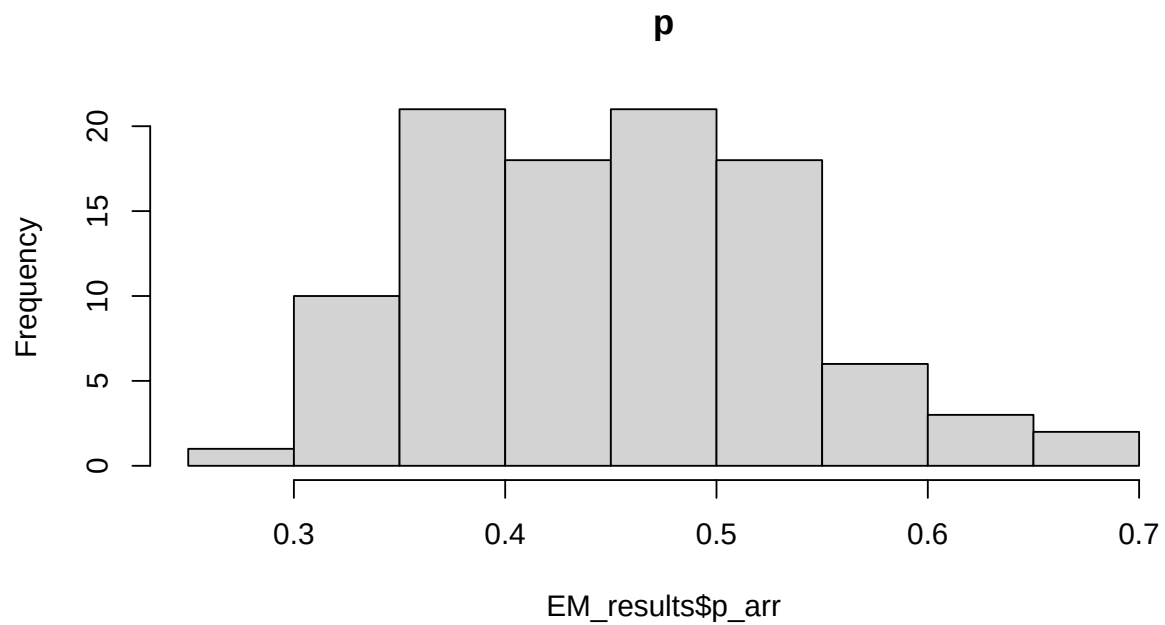
```



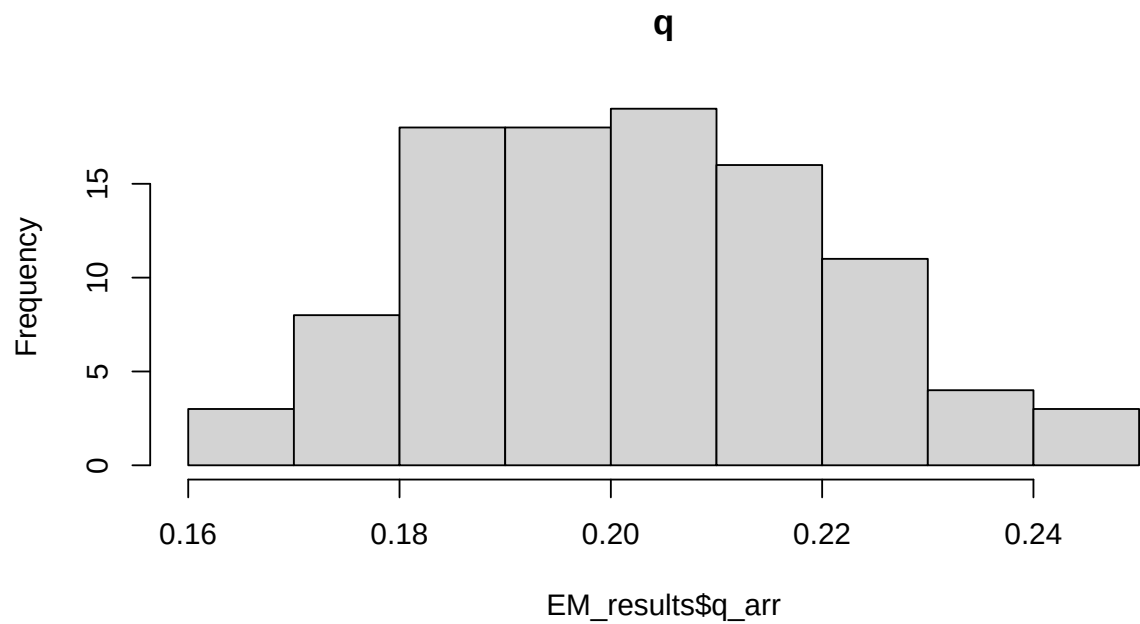
```
hist(EM_results$t_arr, main = "theta")
```



```
hist(EM_results$p_arr, main = "p")
```

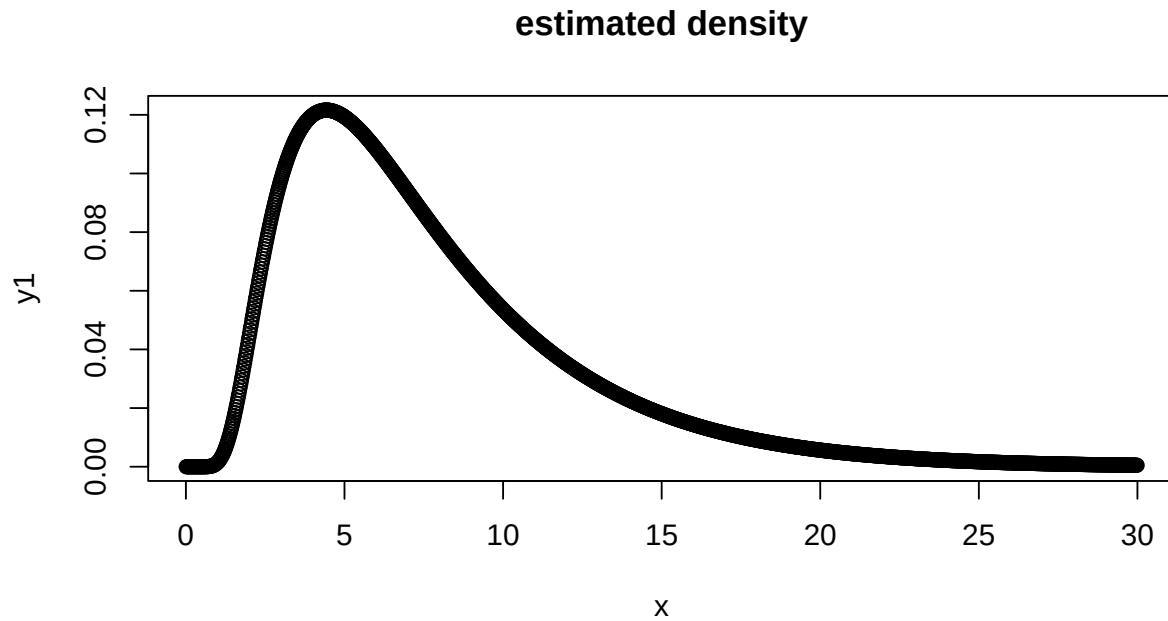


```
hist(EM_results$q_arr, main = "q")
```

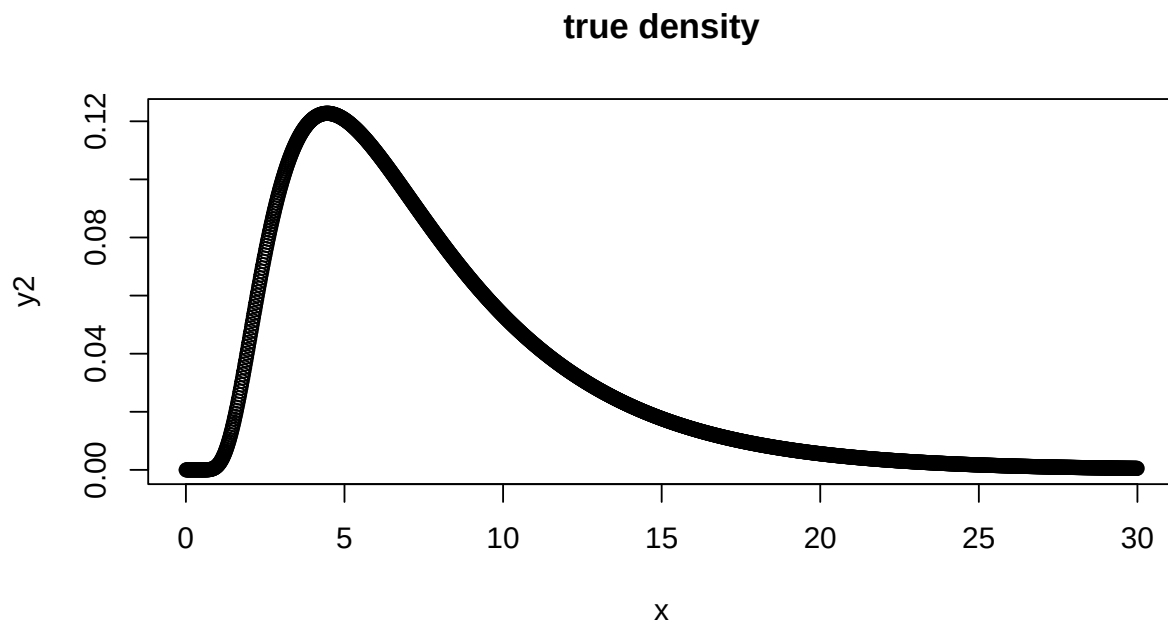


Last we compare the true density function with the estimated one:

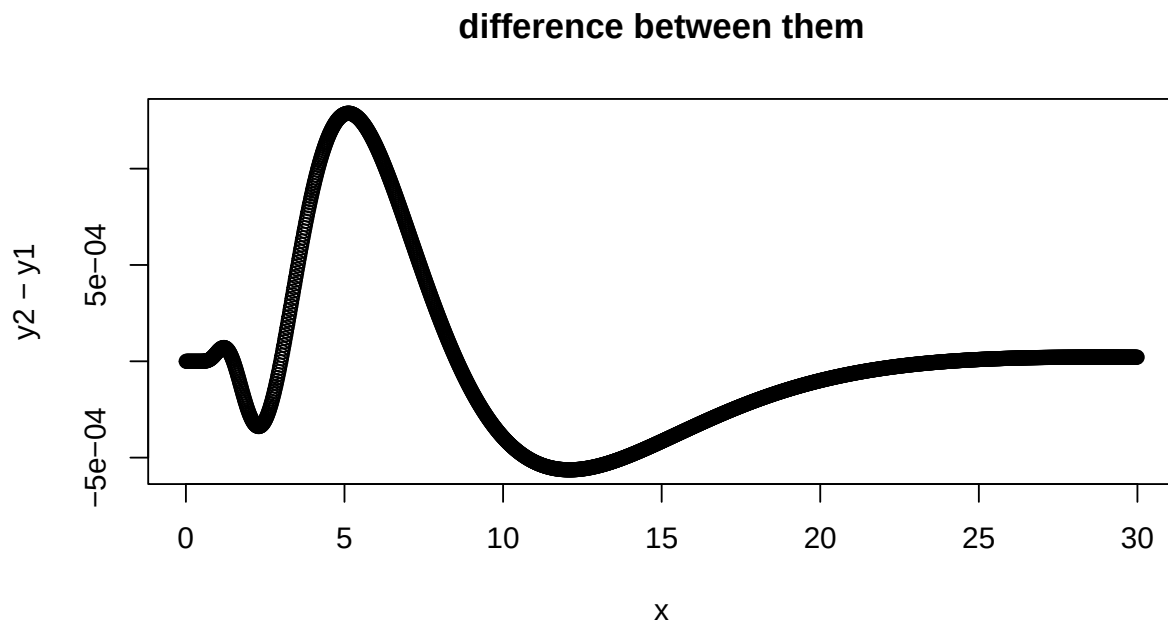
```
x = seq(0.01, 30, 0.02)
y1 = f_GCR(x, EM_results$lambda, EM_results$theta, EM_results$p, EM_results$q)
y2 = f_GCR(x, 3, 2, 0.4, 0.4)
plot(x, y1, main = "estimated density")
```



```
plot(x, y2, main = "true density")
```



```
plot(x, y2 - y1, main = "difference between them")
```



The variance of EM method is acceptable. However, the only issue of it is that the estimation of p and q would be close to initial value, which could result in high bias. I think the reason is, for GCR, one distribution might have many or infinite combination of parameters, and EM would always seek the nearest one.